



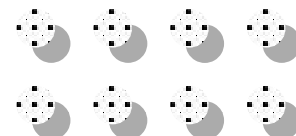
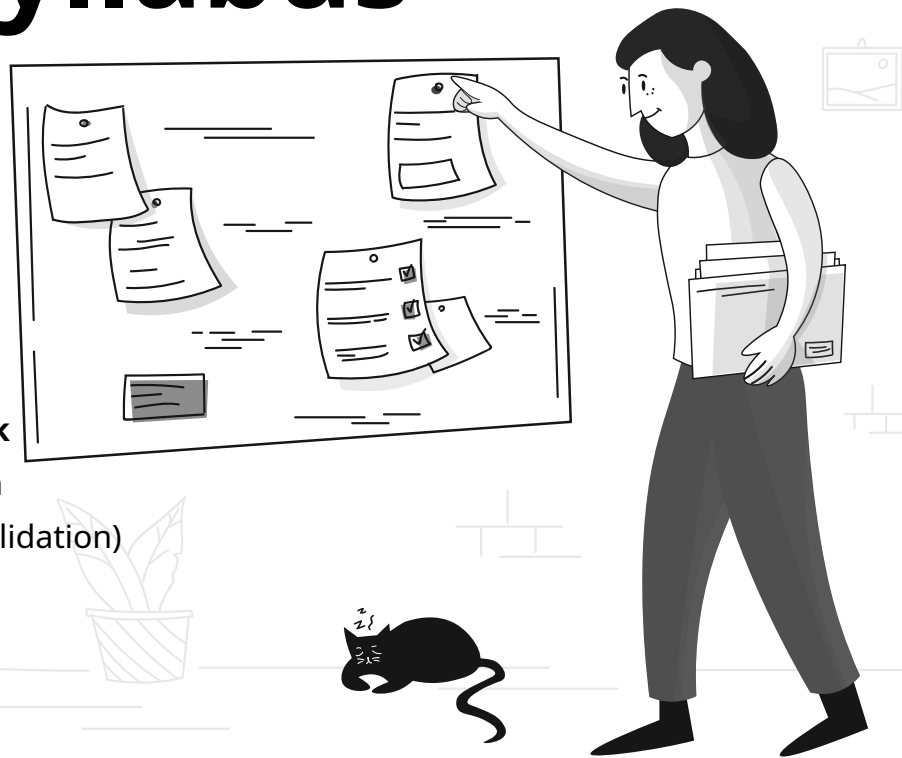
The ML Workflow + Sci-kit Learn Hands on

Eng | Mahmoud Tariq

09/13/2025

Course Syllabus

- Intro to Machine Learning & Environment Setup
- Foundations: Key Linear Algebra & Stats + Hands-on NumPy
- Data Handling & Visualization + Hands-on Pandas
- **The ML Workflow + Linear Regression with Scikit-learn**
 - **Task: Solve session 4's 'task.ipynb' notebook**
- Classification Models (Logistic Regression, KNN) with Scikit-learn
- Model Evaluation & Improvement (Metrics, Overfitting, Cross-Validation)
- Mini-Project: Putting it all together with Scikit-learn
- Introduction to Neural Networks
- Building a Neural Network with a Framework
- Course Project Description + Where to Go From Here



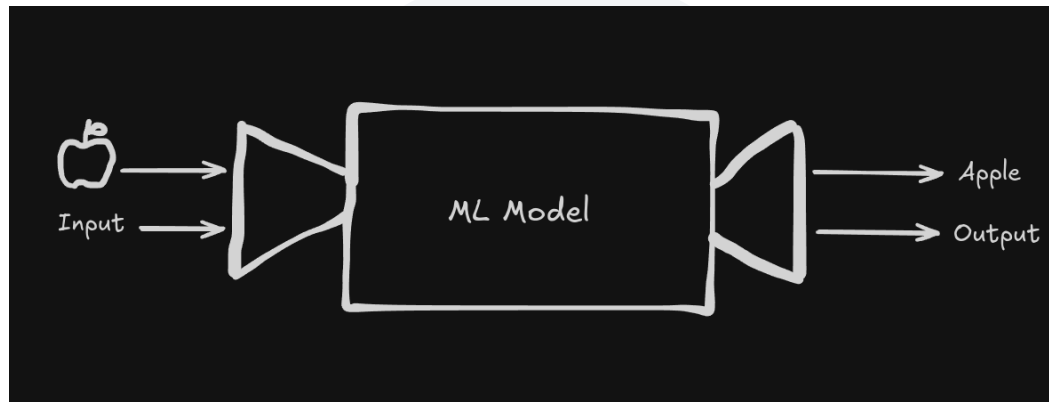
Machine Learning Models

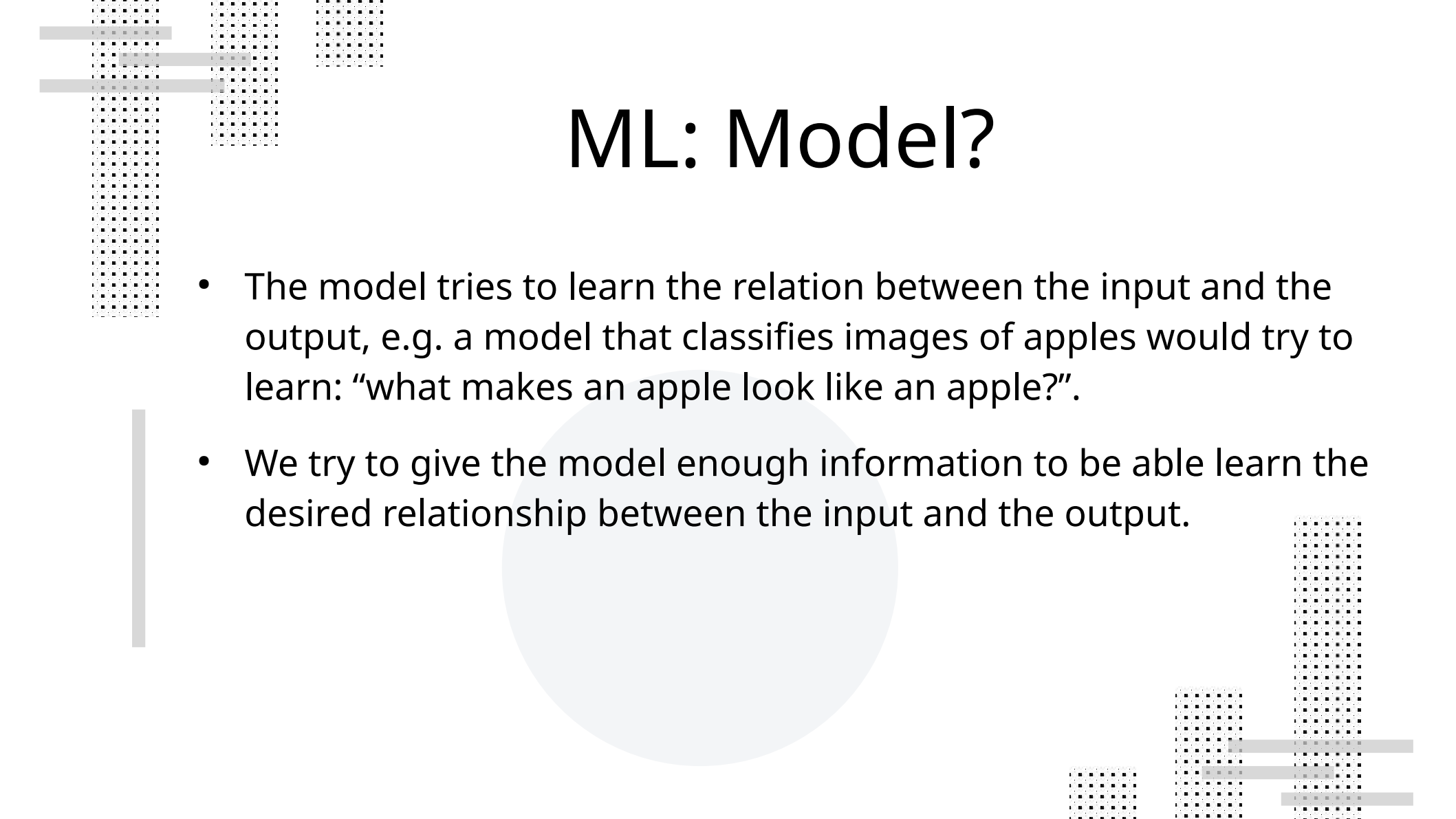
- What is a model?
- The ML Workflow
- Types of Models



ML: Model?

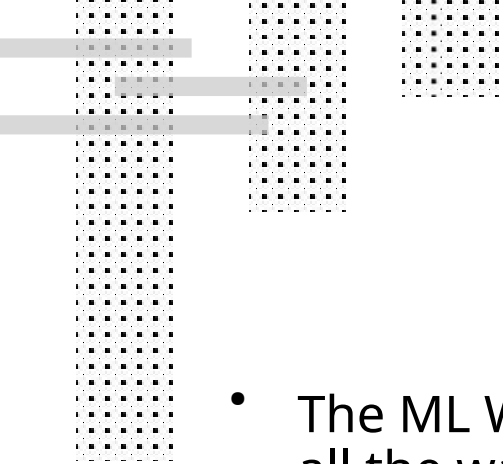
- A Machine Learning model can be thought of as a mapper between input and output:



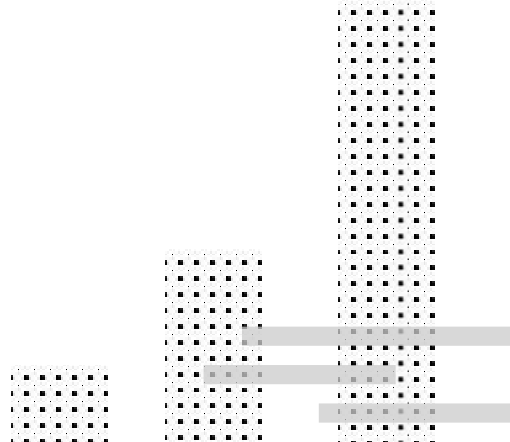


ML: Model?

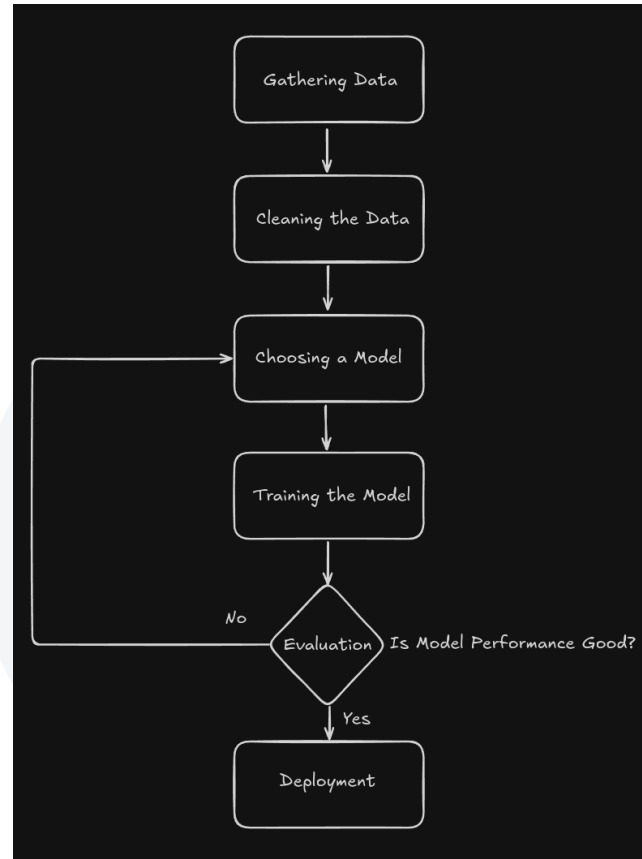
- The model tries to learn the relation between the input and the output, e.g. a model that classifies images of apples would try to learn: “what makes an apple look like an apple?”.
- We try to give the model enough information to be able learn the desired relationship between the input and the output.

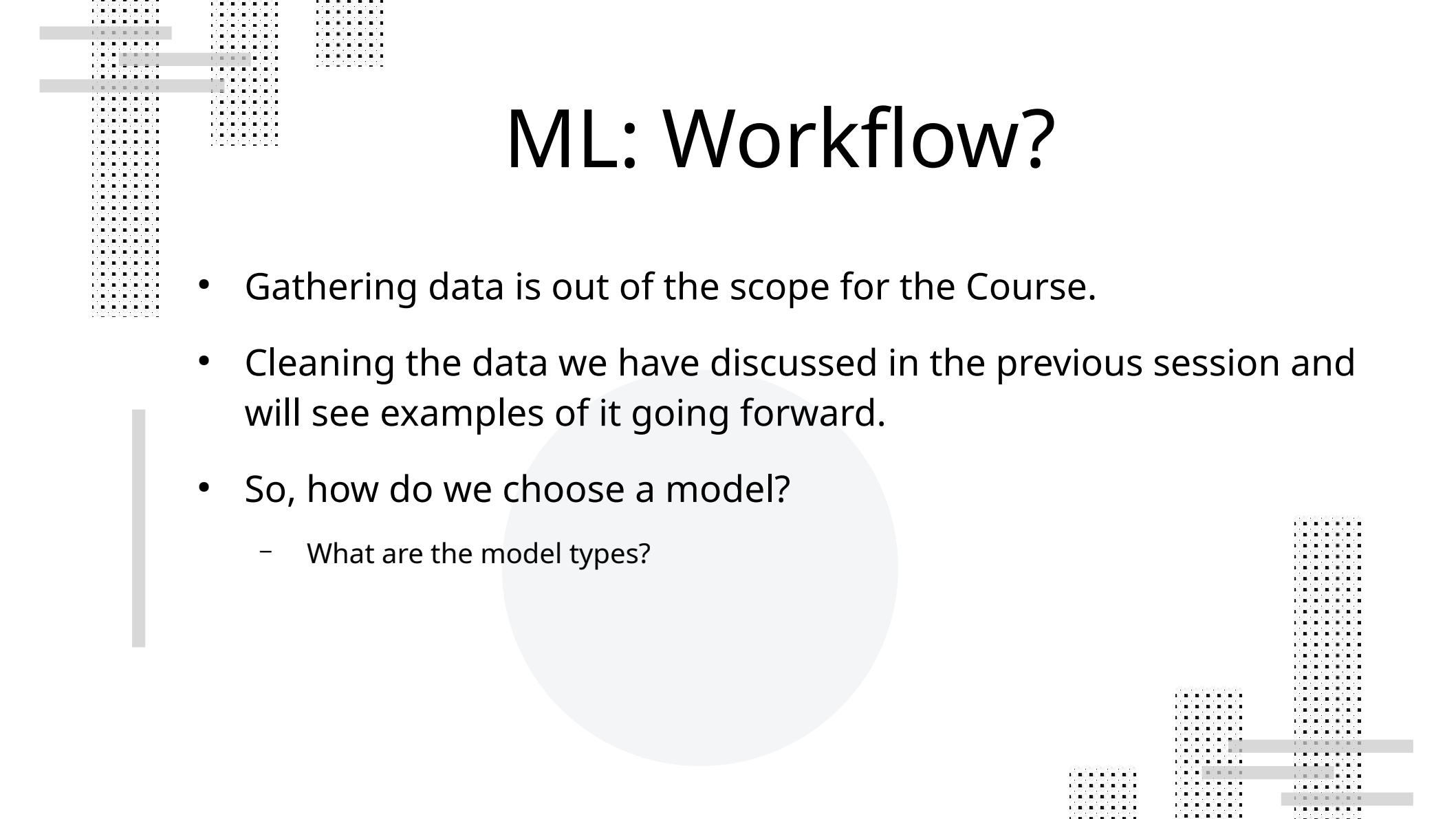


ML: Workflow?

- The ML Workflow: is the process that begins with gathering the data all the way to deploying a functioning Model working in a helpful application.
 - It is mainly composed of:
 - Gathering data.
 - Cleaning the data (as we have learned in the previous session).
 - Choosing the right model.
 - Training the model.
 - Evaluating the model.
 - Deploying the model.
- 

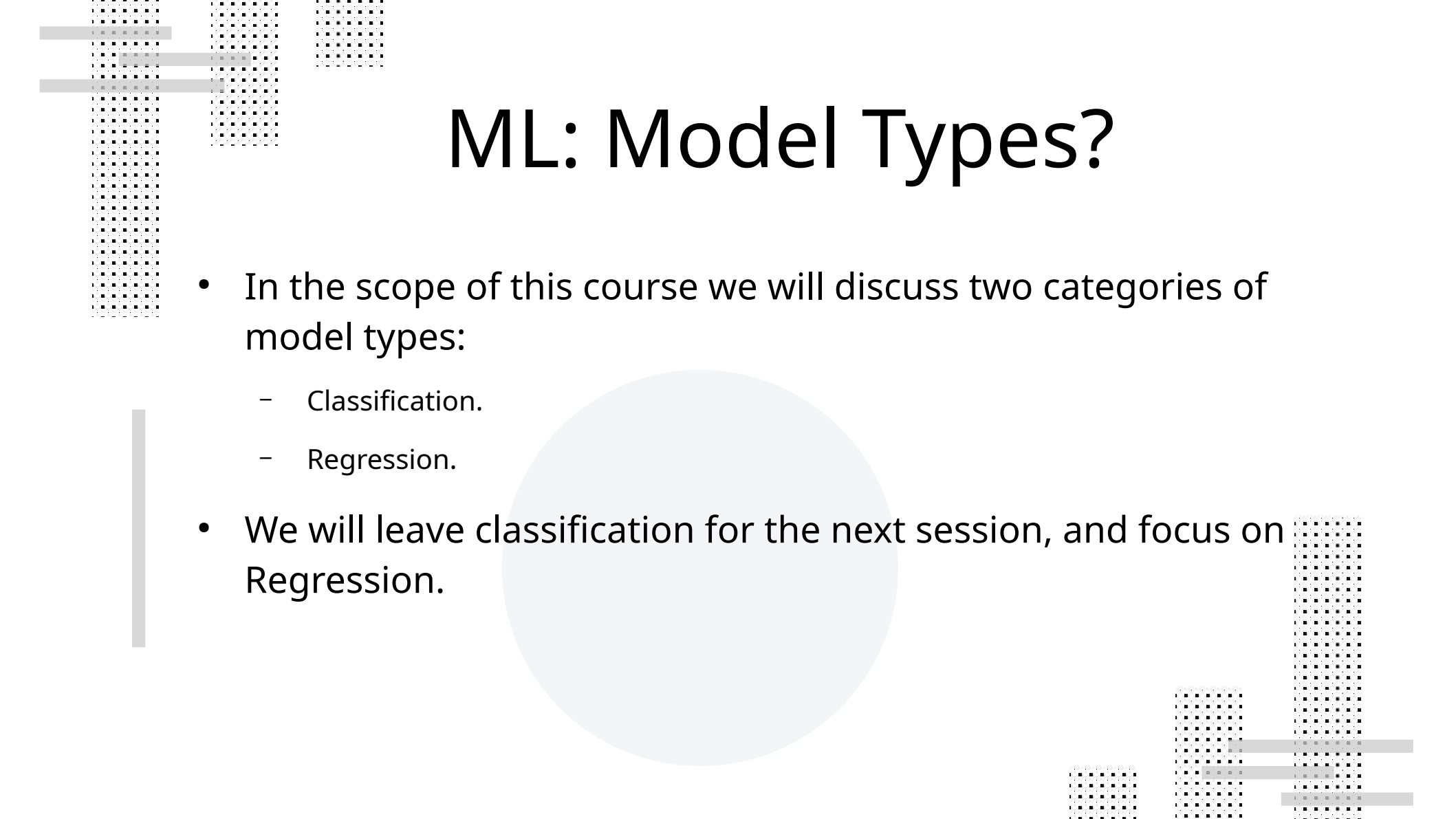
ML: Workflow?





ML: Workflow?

- Gathering data is out of the scope for the Course.
- Cleaning the data we have discussed in the previous session and will see examples of it going forward.
- So, how do we choose a model?
 - What are the model types?



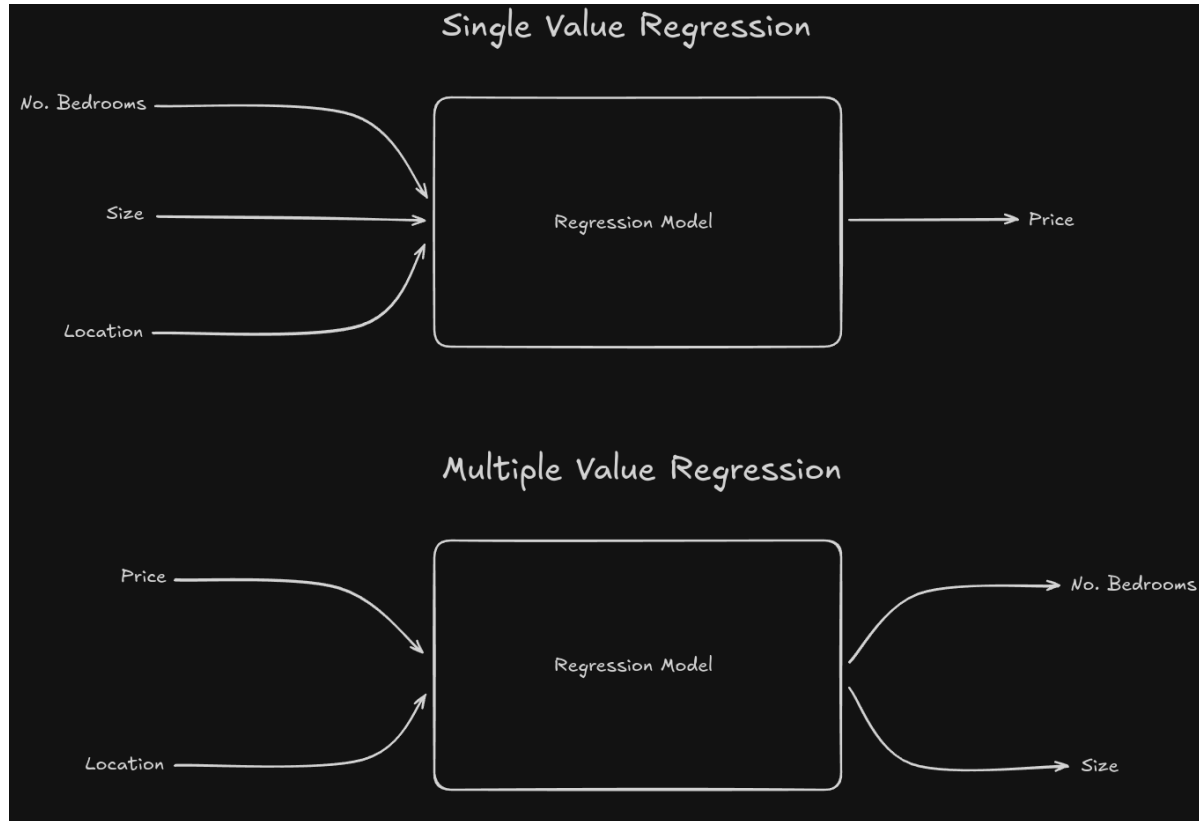
ML: Model Types?

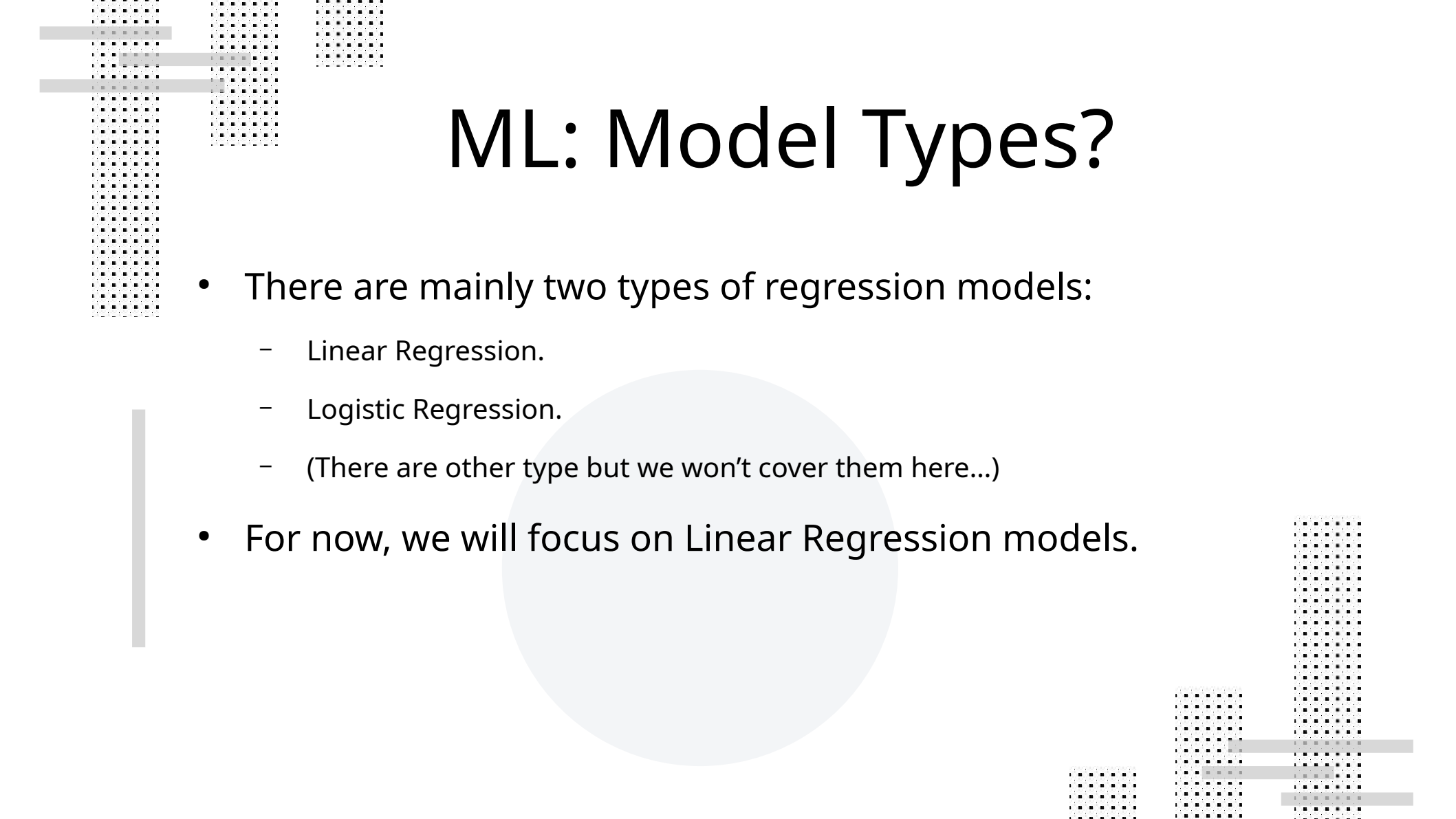
- In the scope of this course we will discuss two categories of model types:
 - Classification.
 - Regression.
- We will leave classification for the next session, and focus on Regression.

ML: Model Types?

- Regression models are ML models capable of regressing to an arbitrary single or multiple values (not selecting from a known set of values).
- e.g. predicting the price of a house based on it's size, No. of bedrooms and location.
- An example of predicting multiple values would be the inverse of the first example, i.e. predicting the number of bedrooms and size of the house based on price and location.

ML: Model Types?





ML: Model Types?

- There are mainly two types of regression models:
 - Linear Regression.
 - Logistic Regression.
 - (There are other type but we won't cover them here...)
- For now, we will focus on Linear Regression models.

Linear Regression

- What is Linear Regression?



Linear Regression?

- a Linear Regression model is a model that can be represented with a 'Linear' equation, generally it is the following equation:

$$y = Xw + b$$

y : is the output of the model.

X : is the input of the model.

w, b : is what the model learns as the relation between X and y

Linear Regression?

- W : is the “weight” matrix that represents the importance of each column to the end result. i.e. an important value like number of rooms in the house would be assigned a higher value $+X$, while a less important feature like the number of bathrooms would be assigned a lower value $-X$.
- B : is the “bias” vector or shift to the perspective of the model. i.e. houses on the same neighborhood would have similar prices, but different neighborhoods have different price ranges, so the bias value allows the model to “shift” perspectives on value ranges.
- Initially we assign these values via a random distribution, and the model “learns” the correct values that yield the best results. e.g. gives the best house price prediction.

Linear Regression?

- A simple example, is the house price prediction problem, but we for simplicity, we will predict only on size:

$$y = Xw + b$$

y : is the output of the model.

X : is the input of the model.

w, b : is what the model learns as the relation between X and y

$$\underset{1 \times 1}{[y]} = \underset{1 \times 1}{[X]} \underset{1 \times 1}{[w]} + \underset{1 \times 1}{[b]}$$

y : in this case is a 1×1 matrix that contains a single value that represents the predicted price

X : also is a 1×1 matrix that represents the size of the house

w : the matrix that represents the importance of each feature (in this case its one feature, thus one value)

b : the bias that allows the model to a wider range of values

Linear Regression?

- More features?, here is what it would look like for three features:

$$\underset{1 \times 1}{[y]} = \underset{1 \times 1}{[X]} \underset{1 \times 1}{[w]} + \underset{1 \times 1}{[b]}$$

y: in this case is a 1x1 matrix that contains a single value that represents the predicted price
X: also is a 1x1 matrix that represents the size of the house
w: the matrix that represents the importance of each feature (in this case its one feature, thus on value)
b: the bias that allows the model to a wider range of values

$$\underset{1 \times 1}{[y]} = \underset{1 \times 3}{[x_1, x_2, x_3]} \underset{3 \times 1}{\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}} + \underset{1 \times 1}{[b]}$$

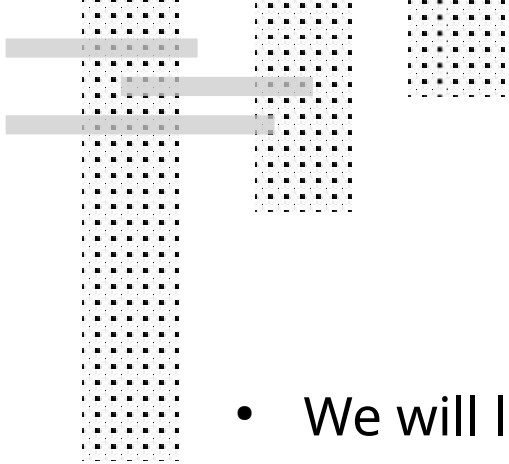
y: remains a 1x1 (i.e a scalar value) output, that represents the price prediction
X: this time the input is a 1x3 matrix, 1 row represents a single instance of the data, and 3 for each feature (e.g. for this example we could say, x1 is the number of rooms, x2 is the size, and x3 the location)
w: remains the importance of each feature but this time, it's 3x1 so the output is 1x1 (we only want price prediction)
(Note: here X is 1x3 and w is 3x1, 1x3 @ 3x1 means the output is 1x1, which is what we want)
b: remains the same

Linear Regression?

- After what we defined earlier, we can generalize for M(number of) features & N outputs with the following equation:

$$\begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}_{1 \times N} = \begin{bmatrix} x_1 & \cdots & x_M \end{bmatrix}_{1 \times M} \begin{bmatrix} w_{11} & \cdots & w_{1N} \\ \vdots & \ddots & \vdots \\ w_{M1} & \cdots & w_{MN} \end{bmatrix}_{M \times N} + \begin{bmatrix} b_1 \\ \vdots \\ b_N \end{bmatrix}_{1 \times N}$$

Note: the change in the Y vector output is changed to $1 \times N$ to denote N outputs are needed, also now the X and w matrices output: $1 \times M @ M \times N = 1 \times N$ matching the needed number of outputs also B bias vector is changed to match it.



Linear Regression?

- We will later explore how the model learns, but now let's code!



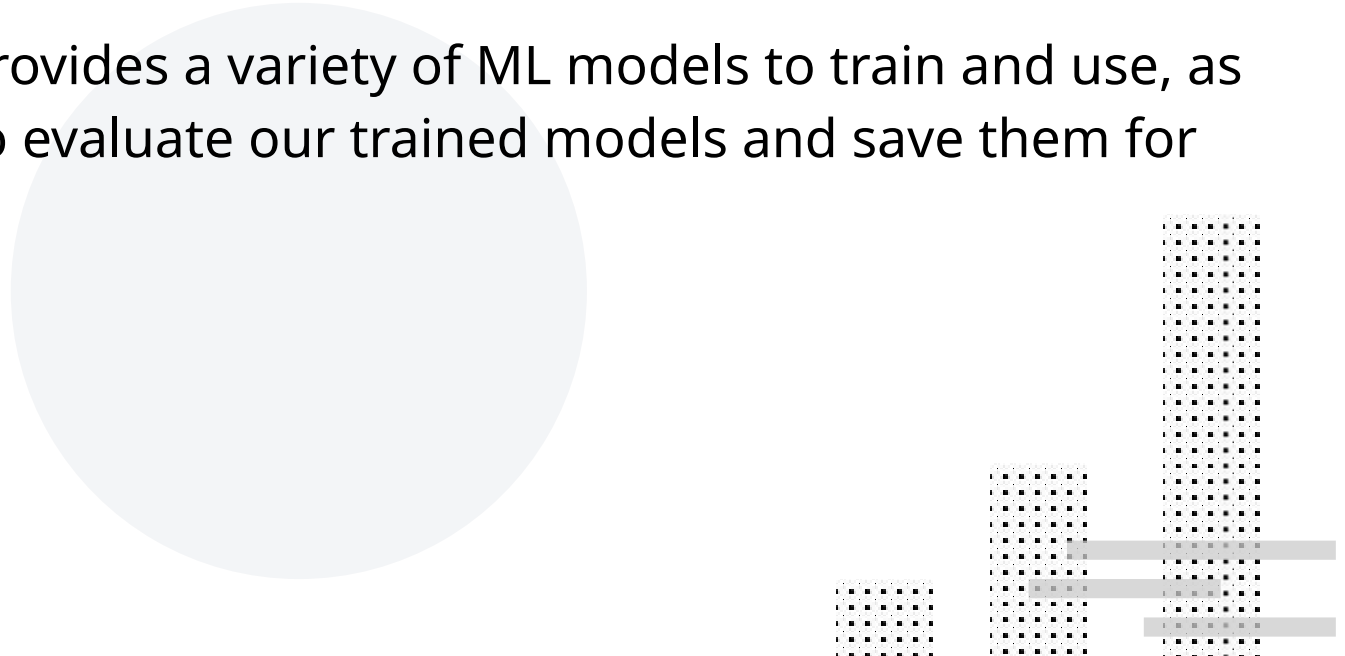
Sci-kit Learn

- What is Sci-kit Learn?
- Sci-kit Learn Hands-on



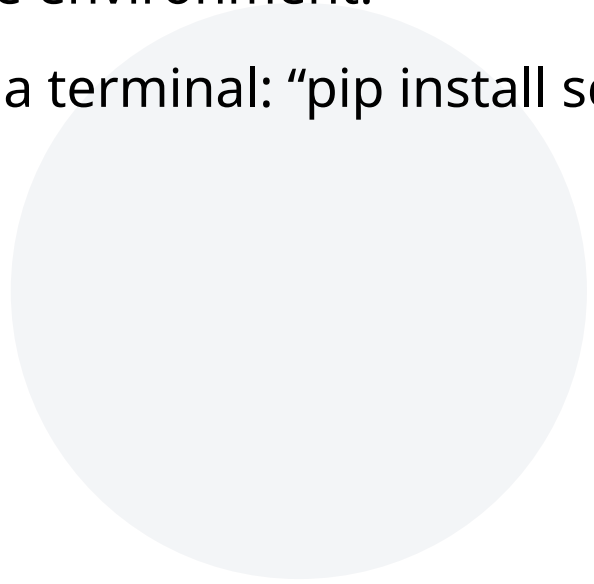
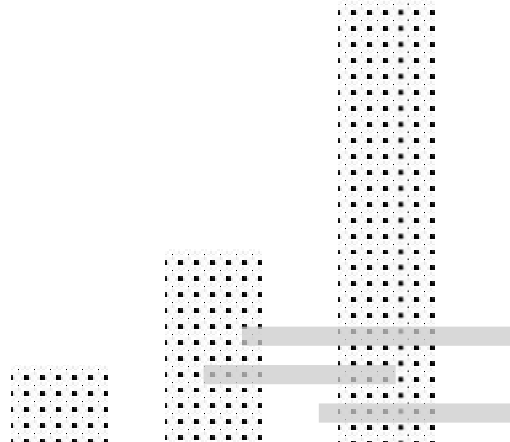


Sci-kit Learn?

- Sci-kit Learn: is a framework like NumPy or Pandas, that provides simple pre-built utilities but in this case, it's ML utilities.
 - Sci-kit Learn provides a variety of ML models to train and use, as well as tools to evaluate our trained models and save them for later use.
- 

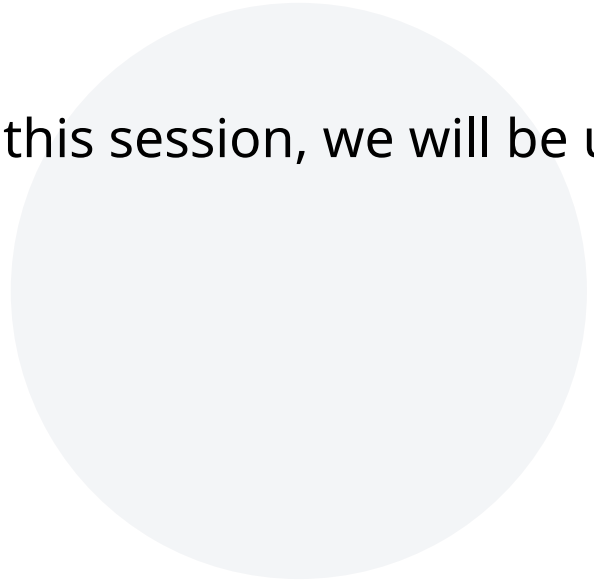
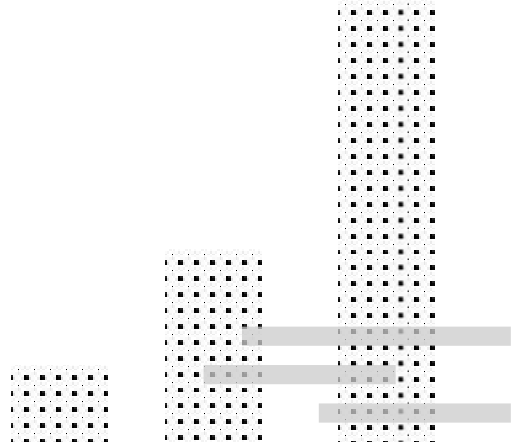


Sci-kit Learn: Installation

- Installing Sci-kit Learn is the same as we did for other packages:
 - Activate the environment.
 - And run in a terminal: “pip install scikit-learn”
- 
- 
- 

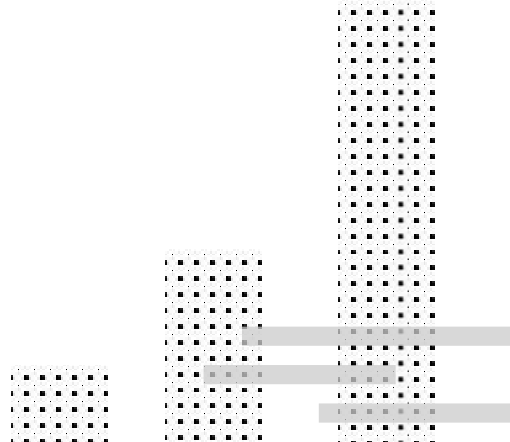


Sci-kit Learn: Preparation

- To begin, we will follow the ML workflow we defined earlier, with cleaning/preping the data as we have the dataset already installed.
 - For the rest of this session, we will be using the california housing dataset.
- 
- 



Sci-kit Learn: Preparation

- Context: The california housing dataset is about the average housing prices in neighborhoods in california, i.e. each row represents a neighborhood with it's details.
- 
- 

Sci-kit Learn: Preparation

- Let's load the dataset:

```
import sklearn # No real standard for importing as we don't use the entire module directly.
import pandas as pd
✓ 0.8s

california_housing = pd.read_csv("california_housing.csv") # load the dataset

california_housing.head() # Take a look at the data
✓ 0.0s
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	ocean_proximity	median_house_value
0	-122.23	37.88	41	880	129.0	322	126	8.3252	NEAR BAY	452600
1	-122.22	37.86	21	7099	1106.0	2401	1138	8.3014	NEAR BAY	358500
2	-122.24	37.85	52	1467	190.0	496	177	7.2574	NEAR BAY	352100
3	-122.25	37.85	52	1274	235.0	558	219	5.6431	NEAR BAY	341300
4	-122.25	37.85	52	1627	280.0	565	259	3.8462	NEAR BAY	342200

Sci-kit Learn: Preparation

- We can already see a more columns than our previous simple examples, let's take a look at their details:

```
california_housing.info() # Check the details of the dataset
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   longitude              20640 non-null  float64
1   latitude               20640 non-null  float64
2   housing_median_age     20640 non-null  int64
3   total_rooms            20640 non-null  int64
4   total_bedrooms         20433 non-null  float64
5   population             20640 non-null  int64
6   households             20640 non-null  int64
7   median_income          20640 non-null  float64
8   ocean_proximity        20640 non-null  object
9   median_house_value     20640 non-null  int64
dtypes: float64(4), int64(5), object(1)
memory usage: 1.6+ MB
```

Notice the missing values

Sci-kit Learn: Preparation

- We can notice already the missing values in 'total_bedrooms' column, let's start with handling that.
- We have:
 - 207 missing values.
 - Looking at the min, max values, there is a stark difference.

```
sum(california_housing.total_bedrooms.isnull()) # Calculate the number of missing values
✓ 0.0s
207

california_housing.total_bedrooms.describe()
✓ 0.0s
count    20433.000000
mean      537.870553
std       421.385070
min        1.000000
25%       296.000000
50%       435.000000
75%       647.000000
max      6445.000000
Name: total_bedrooms, dtype: float64
```

Sci-kit Learn: Preparation

- Since the min and max values are clearly 'outliers', we can't simply use the `'.mean()'` method as it would return an incorrect value.
 - An outlier is a value that clearly lies outside the distribution of the other values.
e.g. consider the numbers: (1 ... 10) a value with 100 or -20 would be considered an outlier.
- So what we need to do, is calculate the mean but to exclude the outlier values from it.

Sci-kit Learn: Preparation

- To find the outliers, we use a method called the Inter-Quartile Range (IQR), this is the most common way of doing this.
- Let's define some terms:
 - Quartile: is the range of values that lie in a quarter of total values, mainly Q1, Q2, Q3. e.g. consider values from 1 to 100, Q1 would be the first quarter i.e. the first 25% of total number, from 1 to 25, Q2 would be the first 50% of values, 1 to 50, lastly Q3 which would simply be 1 to 75.
Note: defining Q4 is meaningless since it would be 100% (all) of the data.
 - Lower Bound: is the lowest value which anything lower would be an outlier.
 - Upper Bound: is the highest value which anything higher would be an outlier.

Sci-kit Learn: Preparation

- We mean to find the Upper and Lower bounds of the data via IQR
IQR is calculated as follows:
 - $IQR = Q3 - Q1$
- And to calculate the Upper & Lower bounds:
 - Lower Bound = $Q1 - 1.5 * IQR$
 - Upper Bound = $Q3 + 1.5 * IQR$
- Let's demonstrate this in code.

Sci-kit Learn: Preparation

```
# Let's Start with getting Q1 & Q3:
q1 = california_housing.total_bedrooms.quantile(0.25) # Q1 is the first 25% (0.25) of data.
q3 = california_housing.total_bedrooms.quantile(0.75) # Q3 is the first 75% (0.75) of data.

# calculate the inter-quartile range: IQR = Q3 - Q1
iqr = q3 - q1

# calculate the upper & lower bounds
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

# let's check their values
print(f"Q1: {q1}, Q3: {q3}, IQR: {iqr}")
print(f"Lower Bound: {lower_bound}, Upper Bound: {upper_bound}")
```

```
Q1: 296.0, Q3: 647.0, IQR: 351.0
Lower Bound: -230.5, Upper Bound: 1173.5
```

```
bedrooms = california_housing.total_bedrooms

outliers = bedrooms[(bedrooms < lower_bound) | (bedrooms > upper_bound)]
print(f"Identified Outliers: \n{len(outliers)}\n")

bedrooms_within_range = bedrooms[~((bedrooms < lower_bound) | (bedrooms > upper_bound))]
print(f"Without Outliers: \n{len(bedrooms_within_range)}\n")
```

```
Identified Outliers:
1271
```

```
Without Outliers:
19369
```

```
# Now let's Calculate the correct mean:
correct_mean = bedrooms_within_range.mean()

incorrect_mean = california_housing.total_bedrooms.mean()

# compare it with the incorrect mean:
print(f"Correct mean: {correct_mean}, Incorrect mean: {incorrect_mean}")
```

```
Correct mean: 458.0337125561006, Incorrect mean: 537.8705525375618
```

Sci-kit Learn: Preparation

- Then we simply fill in the missing values:

```
# Use the ".fillna()" function to fill missing values, use "inplace=True" so we don't have to assign again.
california_housing.total_bedrooms.fillna(correct_mean, inplace=True)

california_housing.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   longitude              20640 non-null  float64
1   latitude               20640 non-null  float64
2   housing_median_age     20640 non-null  int64
3   total_rooms            20640 non-null  int64
4   total_bedrooms         20640 non-null  float64
5   population             20640 non-null  int64
6   households              20640 non-null  int64
7   median_income          20640 non-null  float64
8   ocean_proximity        20640 non-null  object
9   median_house_value     20640 non-null  int64
dtypes: float64(4), int64(5), object(1)
memory usage: 1.6+ MB
```

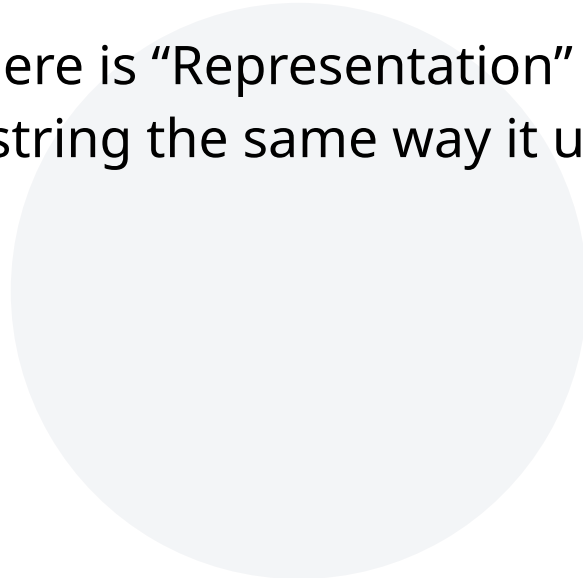
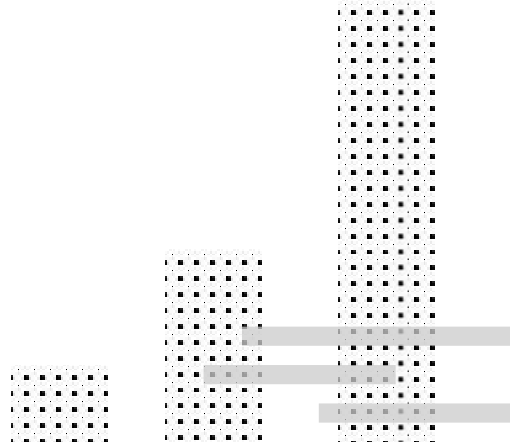

Sci-kit Learn: Preparation

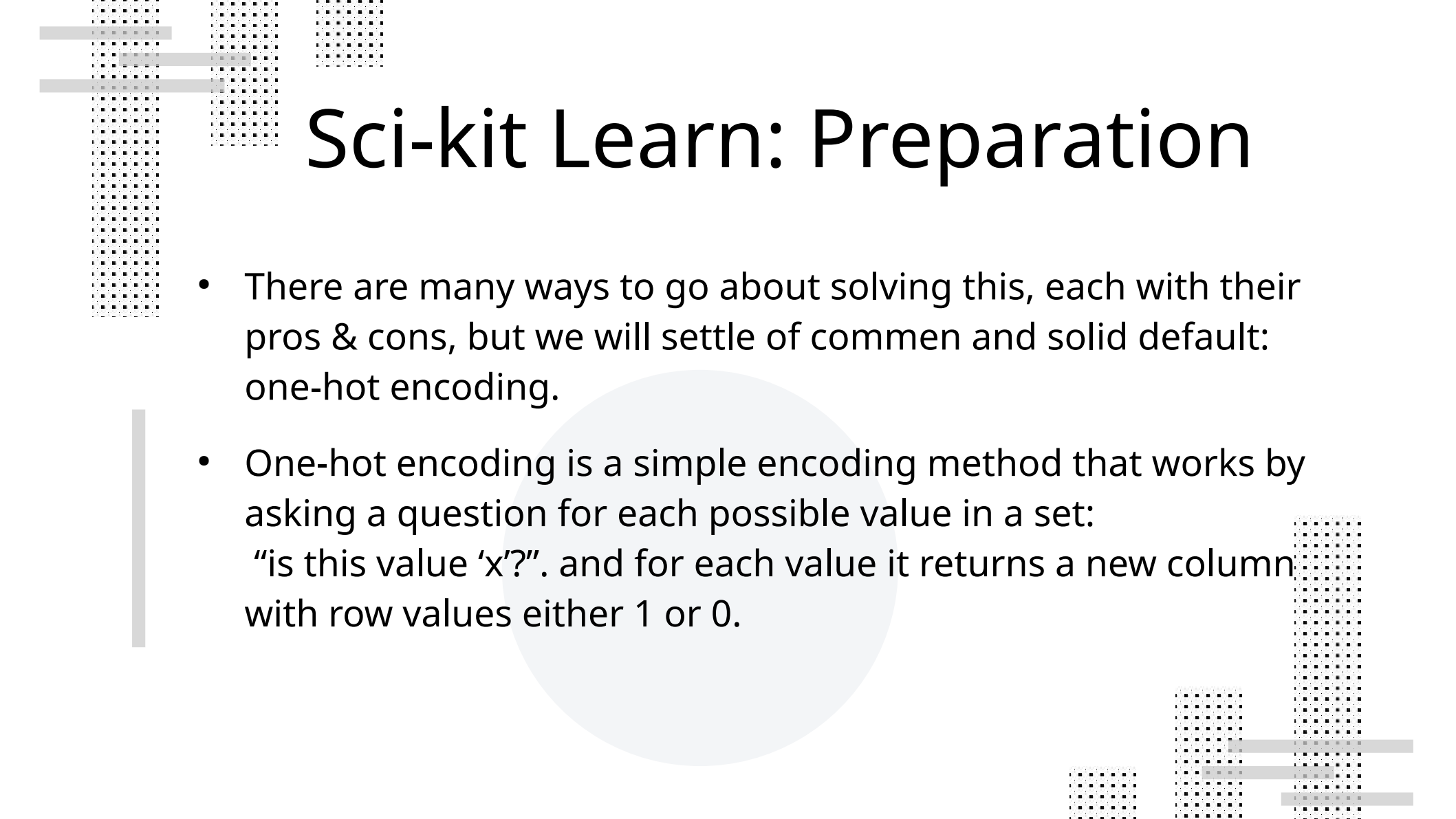
- That's it for the 'total_bedrooms' column, are we ready to move on with our workflow yet?
- Not quite, check out the 'ocean_proximity' column:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   longitude              20640 non-null  float64
1   latitude               20640 non-null  float64
2   housing_median_age     20640 non-null  int64
3   total_rooms            20640 non-null  int64
4   total_bedrooms        20640 non-null  float64
5   population             20640 non-null  int64
6   households             20640 non-null  int64
7   median_income          20640 non-null  float64
8   ocean_proximity        20640 non-null  object
9   median_house_value     20640 non-null  int64
dtypes: float64(4), int64(5), object(1)
memory usage: 1.6+ MB
```



Sci-kit Learn: Preparation

- It's type is 'object' i.e. a string, but our defined equation works for numbers. How do we deal with this?
 - The problem here is “Representation” how do we make the model understand a string the same way it understands a number?
- 
- 



Sci-kit Learn: Preparation

- There are many ways to go about solving this, each with their pros & cons, but we will settle of commen and solid default: one-hot encoding.
- One-hot encoding is a simple encoding method that works by asking a question for each possible value in a set: “is this value ‘x’?”. and for each value it returns a new column with row values either 1 or 0.

Sci-kit Learn: Preparation

- Example:

```
colors = {"colors":["Blue", "Red", "Green", "Blue", "Blue", "Green", "Red"]} # let's define a dataset with one column that has string values.
colors_column = pd.Series(colors['colors']) # turn it into a 'pd.Series' object.
colors_column
```

✓ 0.0s

0	Blue
1	Red
2	Green
3	Blue
4	Blue
5	Green
6	Red

dtype: object

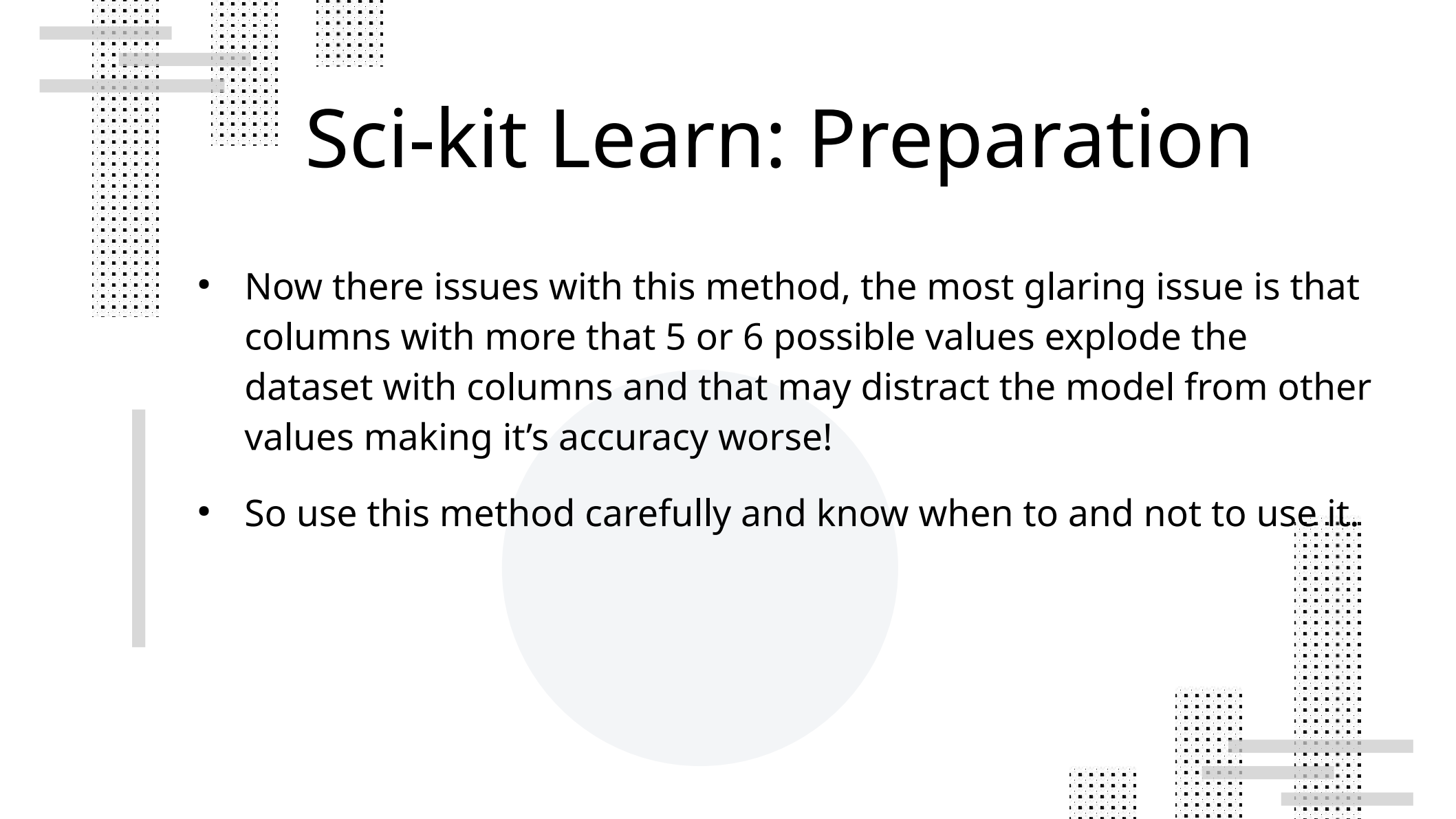
```
one_hot_encoded_colors = pd.get_dummies(colors_column) # Use the "pd.get_dummies" function to one-hot encode columns
one_hot_encoded_colors # Note: each value now is a column, and their values are either true or false (1 or 0).
```

✓ 0.0s

	Blue	Green	Red
0	True	False	False
1	False	False	True
2	False	True	False
3	True	False	False
4	True	False	False
5	False	True	False
6	False	False	True

Sci-kit Learn: Preparation

- Now with the extra 'value' columns the model can deduce the value based on which column has a true or 1 value.
 - (e.g. for the colors dataset, a row with {"Blue": True, "Red": False, "Green": False}, the model will know it is of category "Blue")
 - Note: the model doesn't know column labels. i.e. it's get the value from the column name, instead it defines in it's "thinking" three categories and fits the rows to each.



Sci-kit Learn: Preparation

- Now there are issues with this method, the most glaring issue is that columns with more than 5 or 6 possible values explode the dataset with columns and that may distract the model from other values making its accuracy worse!
- So use this method carefully and know when to and not to use it.

Sci-kit Learn: Preparation

- Let's get back to our "ocean_proximity" column, can we use one-hot encoding here?
- Yes, there are only 5 possible values in the column:

```
len(california_housing.ocean_proximity.unique()) # calculate the number of values in the column
✓ 0.0s
5
```

Sci-kit Learn: Preparation

- Let's clean it and add it to our dataframe object:

```
# We need to get the column, then remove it so there is no duplication and strings in the dataframe:
ocean_proximity = california_housing.pop('ocean_proximity') # '.pop()' deletes the column and returns it

one_hot_encoded_ocean_proximity = pd.get_dummies(ocean_proximity) # get the one-hot encoded version

one_hot_encoded_ocean_proximity.head()
```

	<1H OCEAN	INLAND	ISLAND	NEAR BAY	NEAR OCEAN
0	False	False	False	True	False
1	False	False	False	True	False
2	False	False	False	True	False
3	False	False	False	True	False
4	False	False	False	True	False

```
# "pd.concat" takes a list of dataframes and joins them into one single dataframe
# passing 'axis=1' means to join the columns together, passing 'axis=0' means to put the rows after each other
cleaned_california_housing = pd.concat([california_housing, one_hot_encoded_ocean_proximity], axis=1)
cleaned_california_housing.head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	<1H OCEAN	INLAND	ISLAND	NEAR BAY	NEAR OCEAN
0	-122.23	37.88	41	880	129.0	322	126	8.3252	452600	False	False	False	True	False
1	-122.22	37.86	21	7099	1106.0	2401	1138	8.3014	359500	False	False	False	True	False
2	-122.24	37.85	52	1467	190.0	496	177	7.2574	352100	False	False	False	True	False
3	-122.25	37.85	52	1274	235.0	558	219	5.6431	341300	False	False	False	True	False
4	-122.25	37.85	52	1627	280.0	565	259	3.8462	342200	False	False	False	True	False

Sci-kit Learn: Preparation

- And the last step in preparation is to split the data into X (input) and y (output):

```
y = cleaned_california_housing.pop("median_house_value") # The median price is what we want to predict so we remove it
X = cleaned_california_housing # The rest of the columns are features we will use.
```

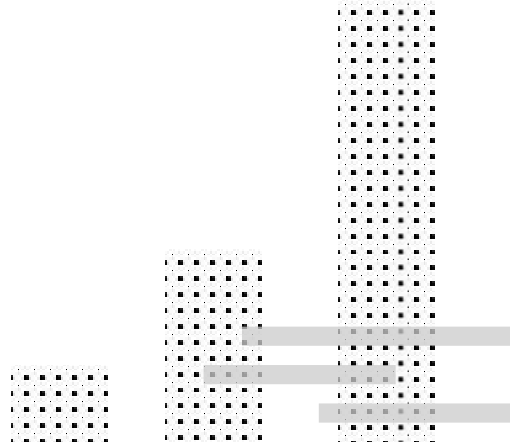
```
X.shape, y.shape
```

```
✓ 0.0s
```

```
((20640, 13), (20640,))
```



Sci-kit Learn: Model Selection

- In the workflow, the next step is model selection. The problem at hand requires predicting a single continuous value which means the best suited model for this is a Regression model!
- 
- 

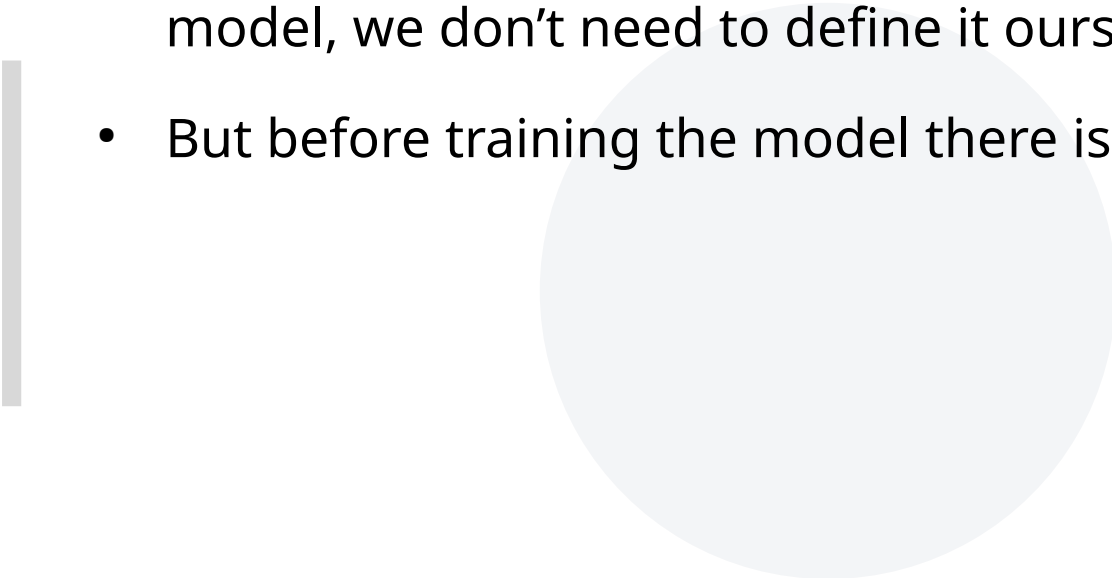
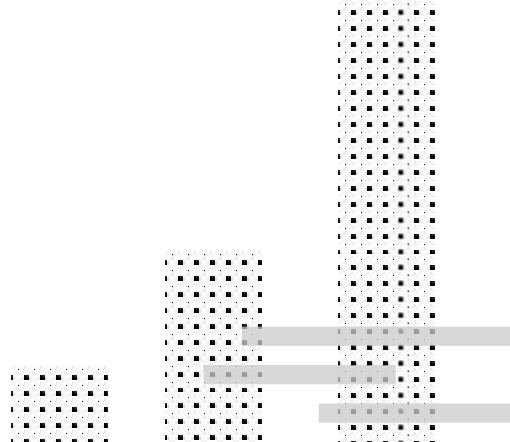
Sci-kit Learn: Model Selection

- Sci-kit Learn provide a variety of regression models, but we will use the “LinearRegression” model:

```
from sklearn.linear_model import LinearRegression, LogisticRegression # you can experiment with the logistic model if you want.  
model = LinearRegression()
```



Sci-kit Learn: Model Selection

- That's it (for creating the model at least)! Sci-kit learn uses the equation we defined earlier underneath to train and define the model, we don't need to define it ourselves!
 - But before training the model there is a missing essential step.
- 
- 

Sci-kit Learn: Train & Test Data

- To correctly evaluate the model later on, we need to test it on data it never saw before. Since if we feed the data it was trained on it will get a very high false score.
- So how do we split the data? 50:50? 60:40? depending on the amount of data you have you may assign less to the test data, but a generally good split is 80:20, 80% training & 20% test. this allows the model to have enough data to learn from, and enough data to be tested on.

Sci-kit Learn: Train & Test Data

- Sci-kit Learn also provides a utility for this, the “train_test_split” function:

```
from sklearn.model_selection import train_test_split # the function is imported from the "model_selection" module

# splits both X & y into train & test, generating four arrays. Note: the ordering is as shown and doesn't change.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

print(f"Train dataset shapes: {X_train.shape, y_train.shape}")
print(f"Test dataset shapes: {X_test.shape, y_test.shape}")

✓ 0.0s

Train dataset shapes: ((16512, 13), (16512,))
Test dataset shapes: ((4128, 13), (4128,))
```

Sci-kit Learn: Training The Model

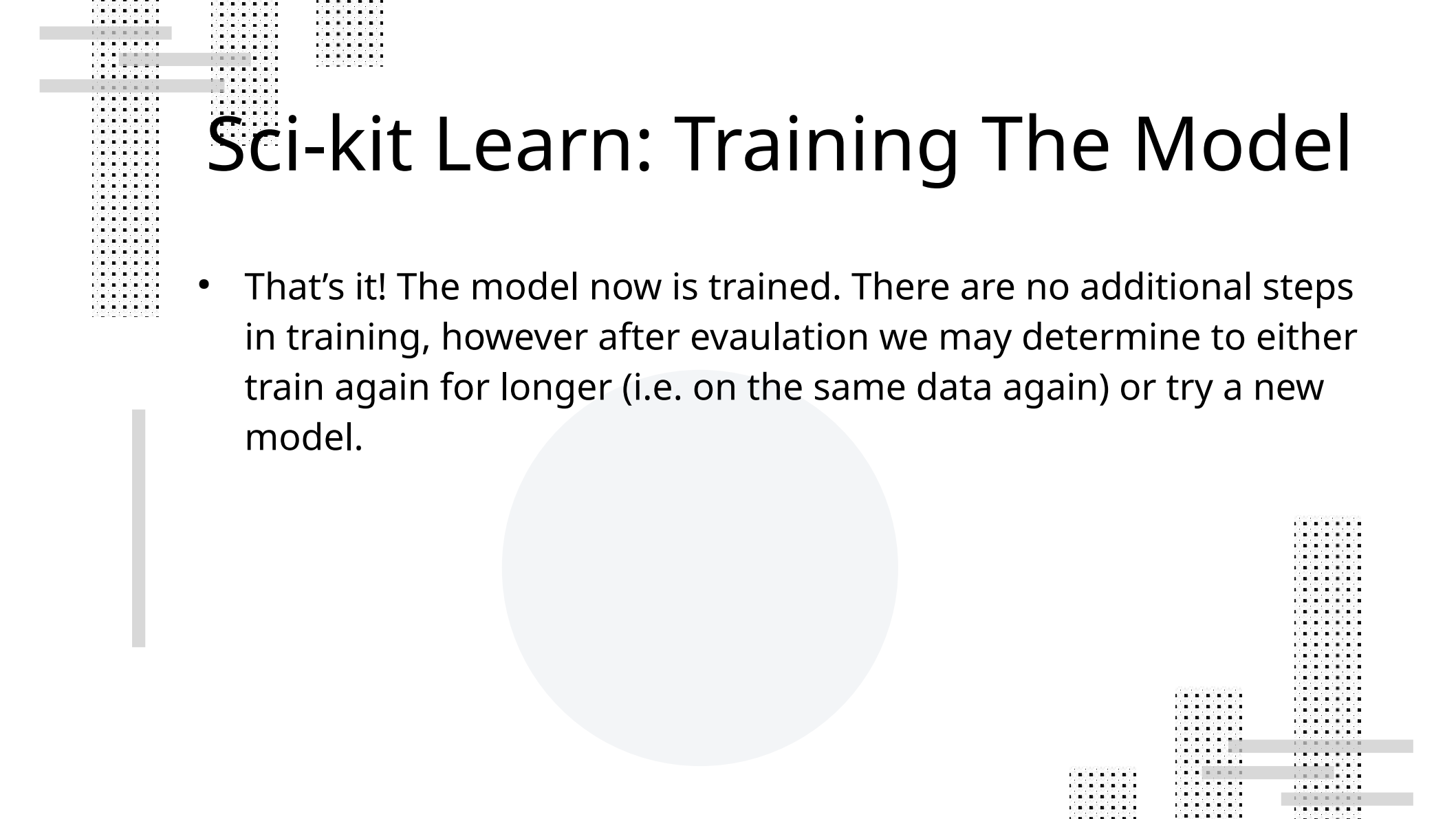
- To train the model in Sci-kit Learn is very simple, and is the same across all models provided by the framework, to do it simply call “.fit(X, y)”:

```
model.fit(X_train, y_train) # simply call '.fit()' with the X input and y output
```

✓ 0.0s

LinearRegression ⓘ ?

▶ Parameters



Sci-kit Learn: Training The Model

- That's it! The model now is trained. There are no additional steps in training, however after evaluation we may determine to either train again for longer (i.e. on the same data again) or try a new model.

Sci-kit Learn: Evaluation

- There are several ways to evaluate a model. In the case of a regression model, we calculate how “off” the model is from each expected value in the data set. (e.g. if the expected value is 10 and the model predicted 20, the model is off by 10)
- To actually calculate how off the model there are several methods and we will explore 2 of them:
 - MAE
 - MSE

Sci-kit Learn: Evaluation

- MAE (Mean Absolute Error): is defined as the absolute value of the difference between the predicted value & the expected value.
- This is the simplest method and is easy to understand (i.e. predicted value is 10, actual value is -10, then mae is 20)

```
from sklearn.metrics import mean_absolute_error # import the function from the 'metrics' module.

# the mean absolute error is defined like:
# mae_score = | real_value - predicted_value |
# this is run across all instances and then an average is taken.

# let's get the model's prediction on the test data via the '.predict()' method which takes only the input data
y_pred = model.predict(X_test)

# then we pass the predicted values to function to get a score of how on average the model is off by.
# Note: the order of passing the values is required to be the real values first then the predicted
mae_score = mean_absolute_error(y_test, y_pred)
mae_score # by how much is our model off?
```

✓ 0.0s

50162.1958766347

Sci-kit Learn: Evaluation

- MSE (Mean Squared Error): is defined as the difference between the predicted value and the actual value, then squared.
- This method is sensitive to outliers, thus if the model predicted a very wrong value it will explode.

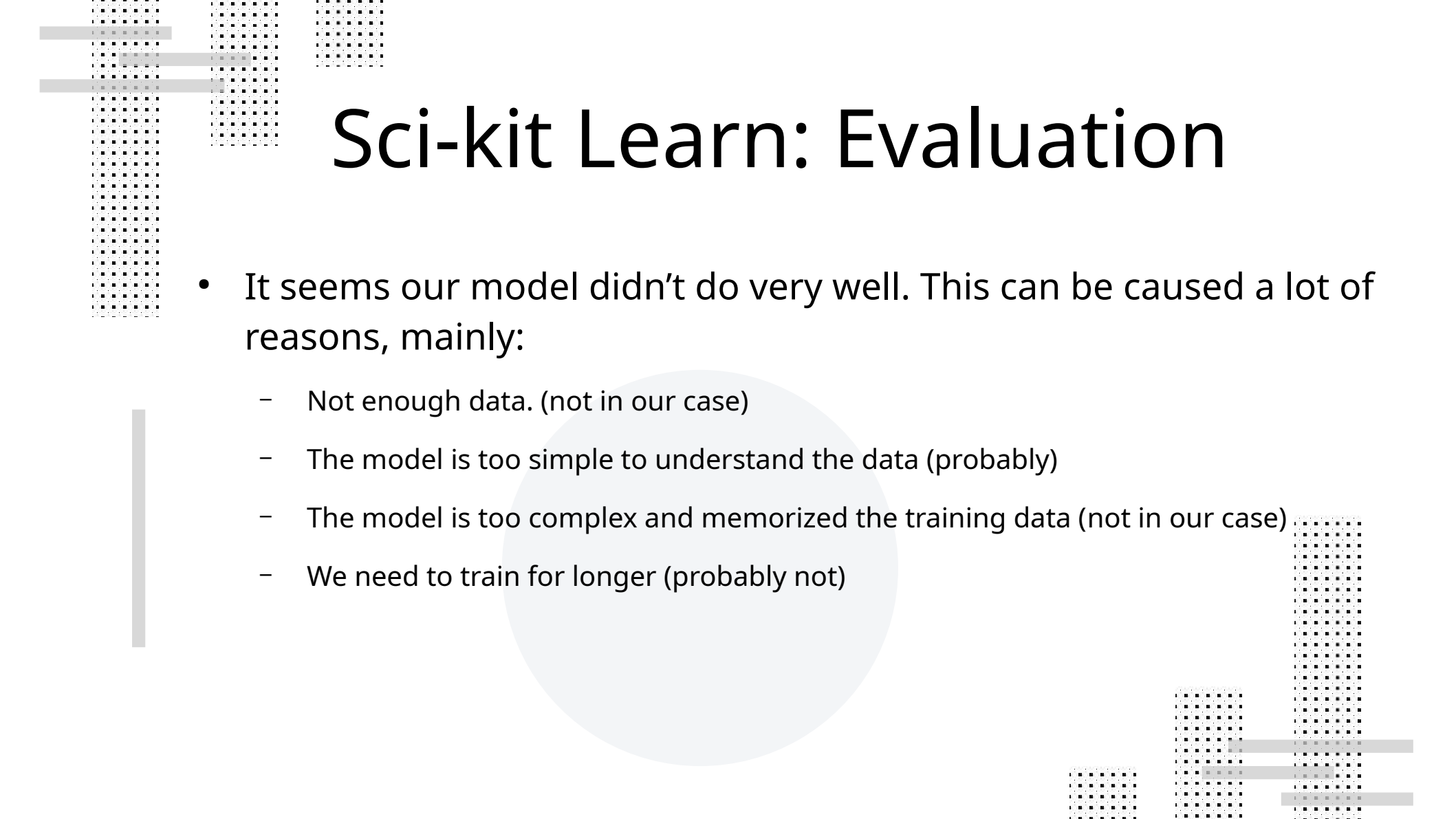
```
from sklearn.metrics import mean_squared_error

# the mean squared error is defined like:
# mse_score = (real_value - predicted_value)**2
# this is run across all instances and then an average is taken.

# we will use the previous prediction.
# Note: the order of passing the values is required to be the real values first then the predicted.
mse_score = mean_squared_error(y_test, y_pred)
mse_score # not really human understandable but is a good metric for outlier detection.
```

✓ 0.0s

4688306714.242042

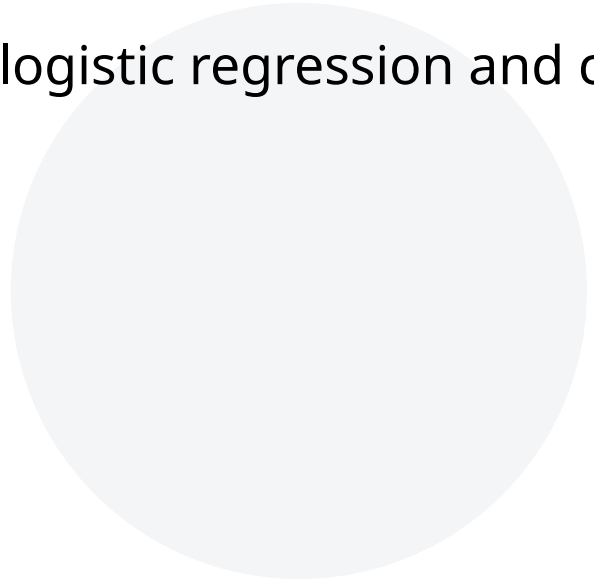
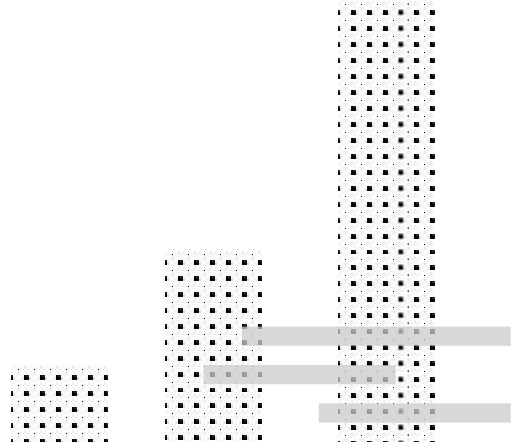


Sci-kit Learn: Evaluation

- It seems our model didn't do very well. This can be caused a lot of reasons, mainly:
 - Not enough data. (not in our case)
 - The model is too simple to understand the data (probably)
 - The model is too complex and memorized the training data (not in our case)
 - We need to train for longer (probably not)



Sci-kit Learn: Conclusion

- A next step would be to use a different model, maybe the 'LogisticRegression' model?
 - We will define logistic regression and classification the next session.
- 
- 

Session Task & What's Next

- Your Task: task.ipynb in session 4 folder
- Next Session:
Classification Models
(Logistic Regression,
KNN) with Scikit-learn



Thank you.

